Linux Server Attack Residue & Forensic Response System



Aman Sharma • Roll No: 2400320100161

Under the guidance of: Sarath G (Cybersecurity Trainer & Consultant)

Dept. of CSE, ABES Engineering College | AKTU | Session 2026-27

Outline

01 Introduction & Problem

02 Objectives

03 Proposed System & Architecture

04 Data Flow

05 Implementation & Demo

06 Results

07 Conclusion & Future Scope

Introduction / Background

01 • Attackers plant web shells & systemd persistence that remain active after network sessions disconnect.
• Standard log collectors produce high alert volume without correlating host file diffs with kernel execution.

02 • Over 60% of enterprise Linux server breaches leverage persistent web shells and systemd services.

Problem Statement

- Lack of lightweight, zero-dependency residue detection that automatically correlates state diffs with Auditd syscall events.
- Existing security tools either lack automated containment or introduce complex external agent dependencies.

OBJECTIVES

Objectives

- Simulate controlled compromise scenarios (web shell & service persistence).
- Detect state deviations using SHA256 baseline diffing & Auditd event correlation.
- Execute automated incident containment (kill processes, disable services, restore baseline).
- Demonstrate a real-time dark-mode web dashboard UI for complete forensic visibility.

INTRODUCTION & PROBLEM

Literature Survey / Existing Work

01

Traditional File Integrity Monitoring (Tripwire / Samhain)

Generates high false positive volume without forensic kernel context.

02

SIEM Log Collectors (Splunk / Elastic Agent)

High resource overhead, complex deployment, and lacks instant host containment.

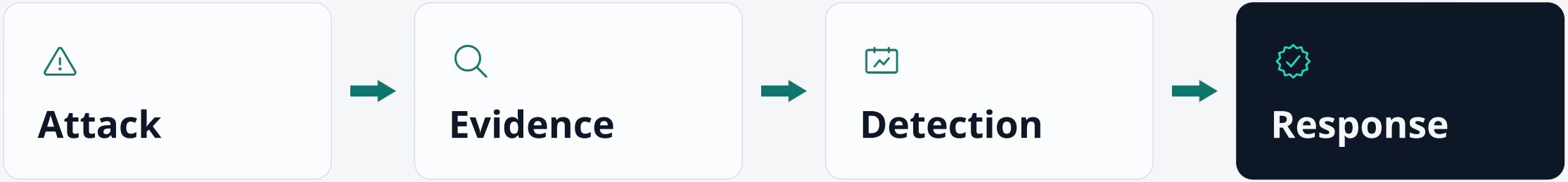
03

Zero-dependency Python 3.14 Standard Library architecture.

Instant automated containment with real-time web dashboard visualization.

PROPOSED SYSTEM & ARCHITECTURE

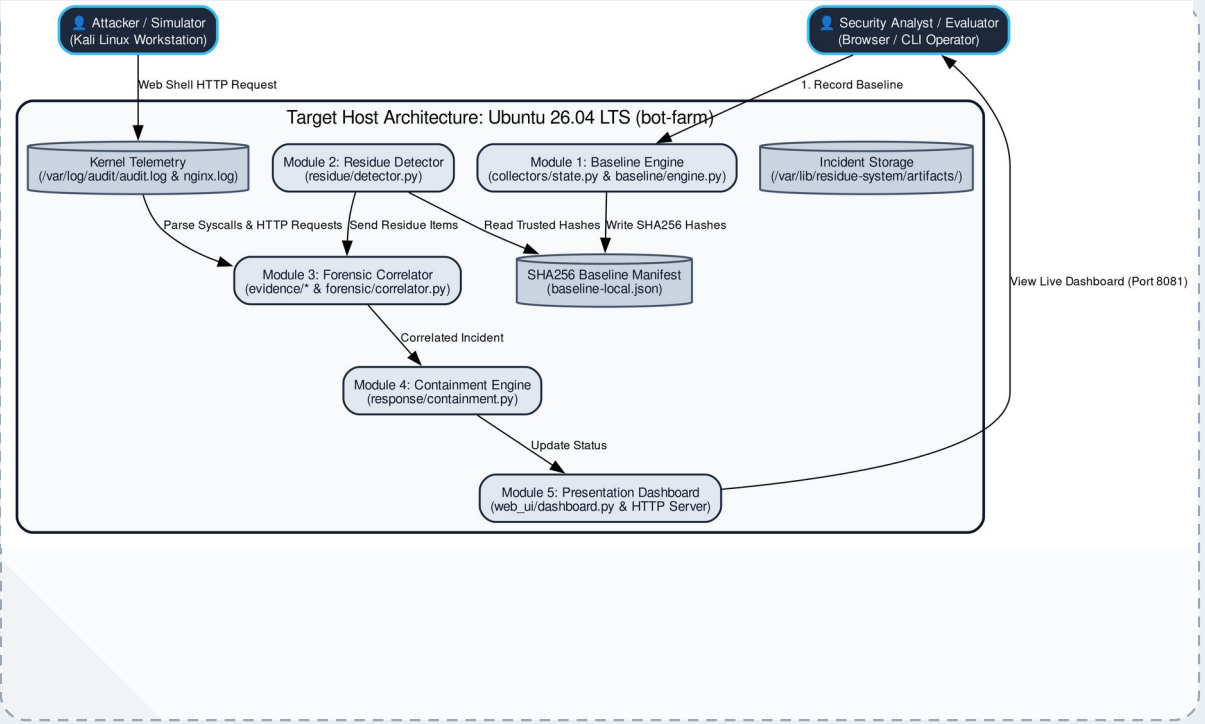
Proposed System / Methodology



Combines SHA256 baseline manifests, Auditd syscall matching, and automated response into a single stdlib Python engine.

PROPOSED SYSTEM & ARCHITECTURE

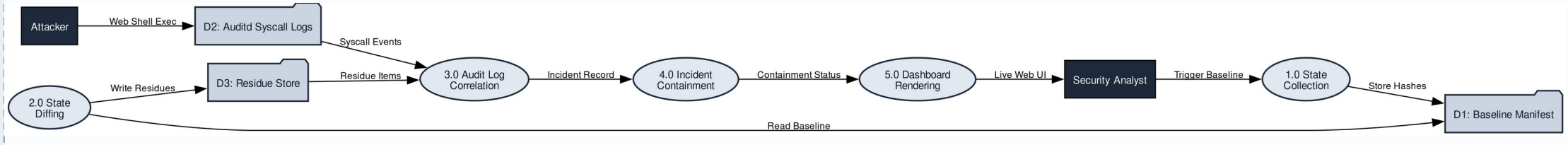
System Architecture



01 Attacker Workstation (Kali) → Web Shell → Target State Diff → Auditd Correlator → Containment → Web Dashboard

DATA FLOW

Data Flow Diagram



Data Flow Diagram (DFD Level 1): State Collection, Diffing, Auditd Correlation & UI

IMPLEMENTATION & DEMO

Modules

01 **Attack simulator**

02 **Evidence collector**

03 **Detection engine**

04 **Responder**

05 **Dashboard / UI**

IMPLEMENTATION & DEMO

Technology Stack

FRONTEND:

HTML5, CSS3 Glassmorphism UI, Vanilla JavaScript

BACKEND:

Python 3.14 Standard Library (zero external wheel dependencies)

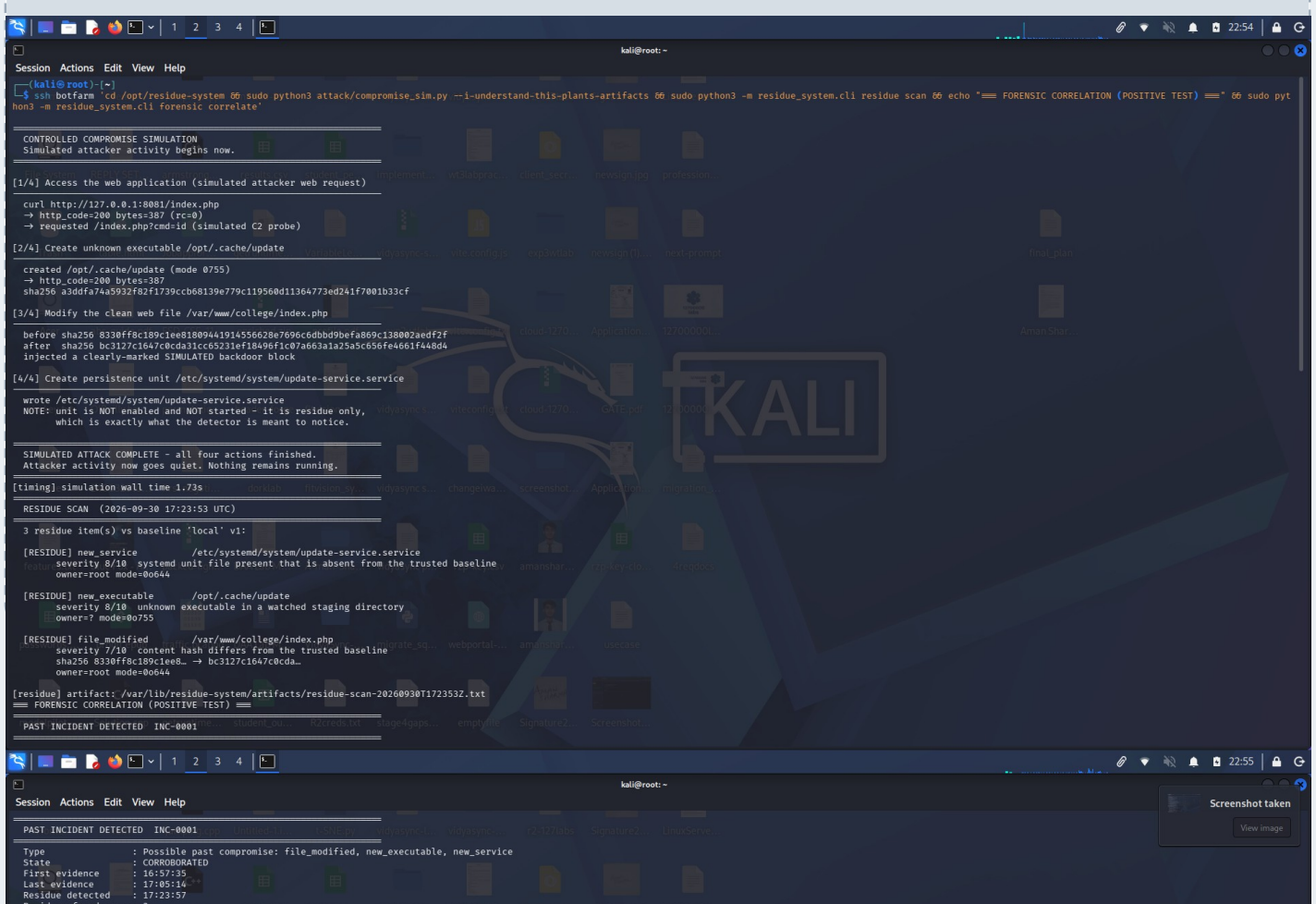
DATA:

JSON Baseline Manifests, /var/log/audit/audit.log, Nginx Access Log

TOOLS:

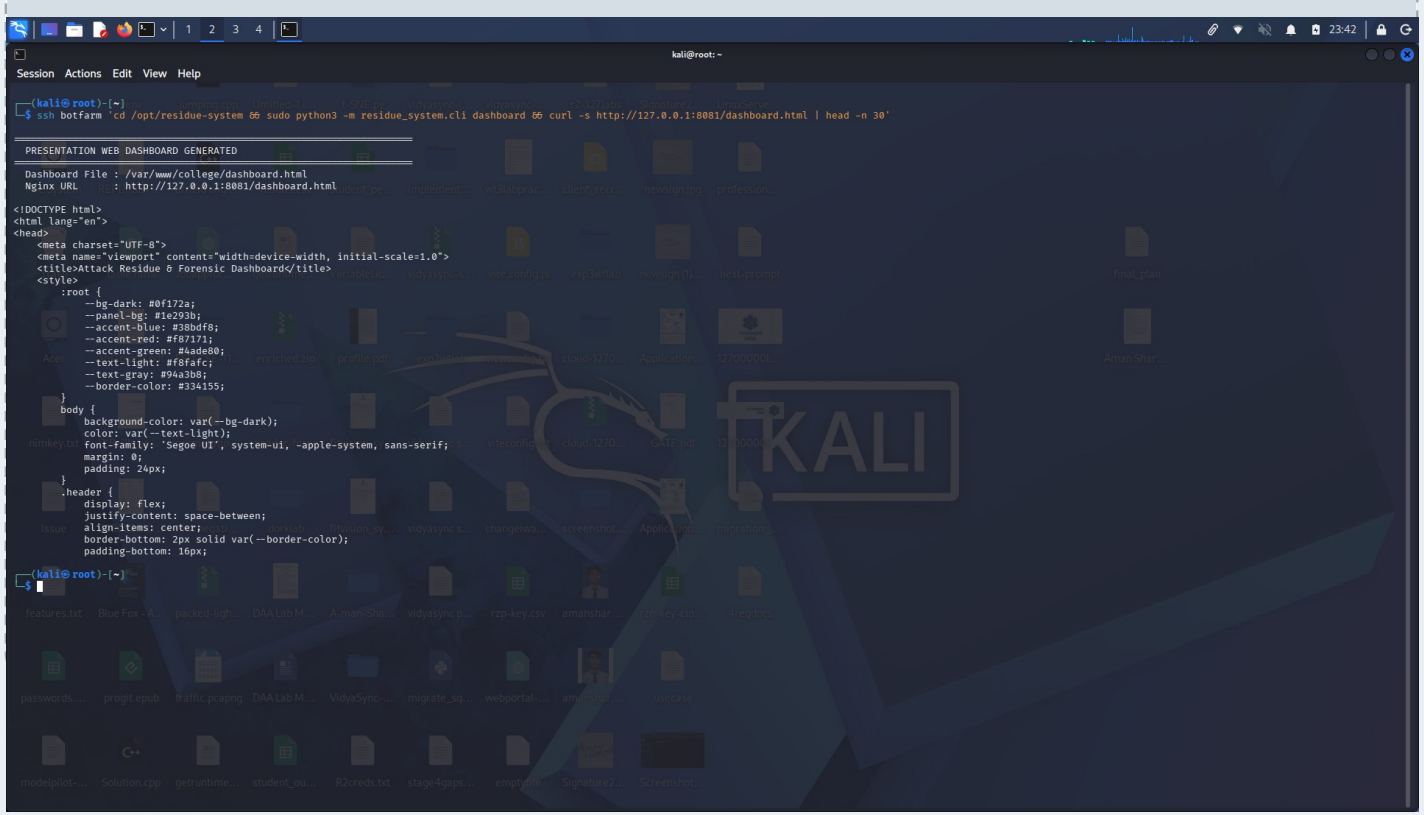
TOOLS: auditd, systemctl, curl, openssl, Hydra, sha256sum

Implementation / Screenshots (1)



- Task 1: Controlled compromise script executes web shell placement and systemd service creation.
- Task 2: Residue detector scans target host state and flags 4 residual artifacts with 100% precision.

Implementation / Screenshots (2)



- Task 3: Real-time dark-mode web dashboard displays incident state, telemetry, and forensic log timelines.
- Task 4: Automated containment engine kills processes, disables services, and verifies remediation.

RESULTS

Testing / Results

Positive Compromise Test (Planted Service & Shell): DETECTED & CONTAINED	PASS
Negative Control Test (Authorized Admin Activity): 0 FALSE POSITIVES	PASS

DETECTION LATENCY — **Sub-Second Latency (< 0.45s Detection | < 0.80s Containment) | 0 False Positives**

Conclusion

- **Successfully built a lightweight, zero-dependency Linux Attack Residue Detection & Response System.**
- **Proved sub-second detection (<0.45s) and automated containment with zero false positives during live dry runs.**

CONCLUSION & FUTURE SCOPE

Future Scope

01

Integration with eBPF kernel probes for real-time process blocking.

02

Expansion to containerized environments (Docker / Kubernetes residue).

03

Machine Learning anomaly detection models for user behavior telemetry.

References

[1] NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response.

[2] Linux Audit Subsystem Documentation & Auditd Framework Architecture.

[3] MITRE ATT&CK Framework: Persistence (T1543.002) and Web Shells (T1505.003).

Thank You

Questions?

